MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: ANALYSIS OF ALGORITHM & 6CS4-02

Branch: CSE 6TH SEM Batch: A&B

Semester: VIth **Session:** 2023-24(even Sem.)

Unit No.: 2 **Date of Submission: 27-02-2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Greedy Method: Overview of the greedy paradigm	2-4	3
2	Example of exact optimization solution	5-8	4
3	Minimum cost spanning tree, Approximate solutions	9-13	5
4	Knapsack problem, Kruskal's algorithm	14-20	7
5	Prim's algorithm for finding minimum cost spanning tree	21-24	4
6	Dijkstra's and Bellman ford algorithm for finding single source shortest paths	25-45	21
7	Huffman coding	46-47	2
8	Activity Selection problem	48-49	2

References Book:

1. T .H .Cormen, C .E .Leiserson, R .L . Rivest "Introduction to Algorithms", 3rd Ed.,PHI, 2011
2. **E. Horowitz, S. Sahni, and S. Rajsekaran, "Fundamentals of Computer Algorithms,"Galgotia Publication**
3. **S.K Basu- Design Methods and Analysis of Algorithms, 2nd Ed., PHI**

FACULTY **HOD** **DEAN ACADEMICS**

UNIT 2

LECTURE 1

Greedy Algorithm

The greedy method is one of the strategies like Divide and conquer used to solve the problems. This method is used for solving optimization problems. An optimization problem is a problem that demands either maximum or minimum results. Let's understand through some terms.

The Greedy method is the simplest and straightforward approach. It is not an algorithm, but it is a technique. The main function of this approach is that the decision is taken on the basis of the currently available information. Whatever the current information is present, the decision is made without worrying about the effect of the current decision in future.

This technique is basically used to determine the feasible solution that may or may not be optimal. The feasible solution is a subset that satisfies the given criteria. The optimal solution is the solution which is the best and the most favorable solution in the subset. In the case of feasible, if more than one solution satisfies the given criteria then those solutions will be considered as the feasible, whereas the optimal solution is the best solution among all the solutions.

Characteristics of Greedy method

The following are the characteristics of a greedy method:

- To construct the solution in an optimal way, this algorithm creates two sets where one set contains all the chosen items, and another set contains the rejected items.
- A Greedy algorithm makes good local choices in the hope that the solution should be either feasible or optimal.

Components of Greedy Algorithm

The components that can be used in the greedy algorithm are:

- **Candidate set:** A solution that is created from the set is known as a candidate set.
- **Selection function:** This function is used to choose the candidate or subset which can be added in the solution.
- **Feasibility function:** A function that is used to determine whether the candidate or subset can be used to contribute to the solution or not.

- **Objective function:** A function is used to assign the value to the solution or the partial solution.
- **Solution function:** This function is used to intimate whether the complete function has been reached or not.

Applications of Greedy Algorithm

- It is used in finding the shortest path.
- It is used to find the minimum spanning tree using the prim's algorithm or the Kruskal's algorithm.
- It is used in a job sequencing with a deadline.
- This algorithm is also used to solve the fractional knapsack problem.

```
1. Algorithm Greedy (a, n)
2. {
3.   Solution : = 0;
4.  for i = 0 to n do
5.  {
6.     x: = select(a);
7.    if feasible(solution, x)
8.    {
9.       Solution: = union(solution , x)
10.   }
11.     return solution;
12. } }
```

The above is the greedy algorithm. Initially, the solution is assigned with zero value. We pass the array and number of elements in the greedy algorithm. Inside the for loop, we select the element one by one and checks whether the solution is feasible or not. If the solution is feasible, then we perform the union.

Let's understand through an example.

Suppose there is a problem 'P'. I want to travel from A to B shown as below:

P : A → B

The problem is that we have to travel this journey from A to B. There are various solutions to go from A to B. We can go from A to B by **walk, car, bike, train, aeroplane**, etc. There is a constraint in the journey that we have to travel this journey within 12 hrs. If I go by train or aeroplane then only, I can cover this distance within 12 hrs. There are many solutions to this problem but there are only two solutions that satisfy the constraint.

If we say that we have to cover the journey at the minimum cost. This means that we have to travel this distance as minimum as possible, so this problem is known as a minimization problem. Till now, we have two feasible solutions, i.e., one by train and another one by air. Since travelling by train will lead to the minimum cost so it is an optimal solution. An optimal solution is also the feasible solution, but providing the best result so that solution is the optimal solution with the minimum cost. There would be only one optimal solution.

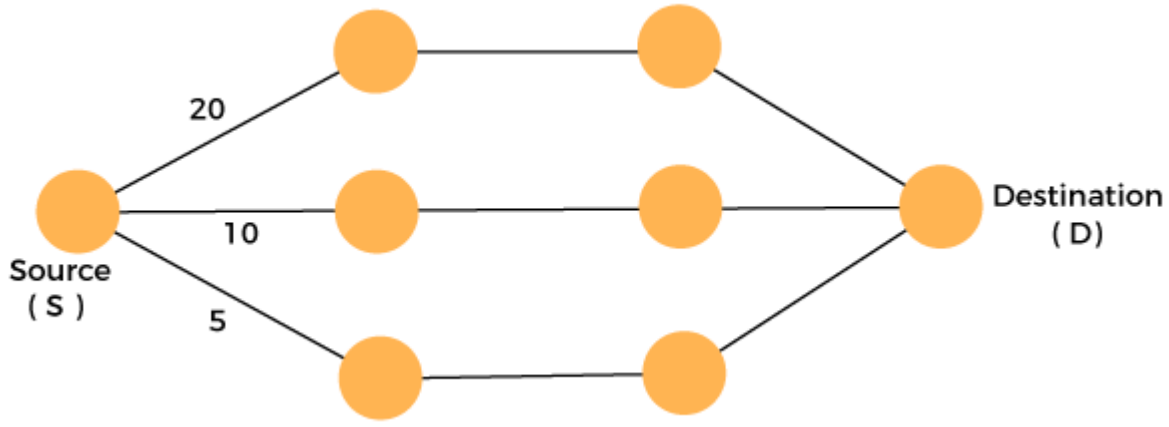
The problem that requires either minimum or maximum result then that problem is known as an optimization problem. Greedy method is one of the strategies used for solving the optimization problems.

Disadvantages of using Greedy algorithm

Greedy algorithm makes decisions based on the information available at each phase without considering the broader problem. So, there might be a possibility that the greedy solution does not give the best solution for every problem.

It follows the local optimum choice at each stage with a intend of finding the global optimum. Let's understand through an example.

Consider the graph which is given below:



We have to travel from the source to the destination at the minimum cost. Since we have three feasible solutions having cost paths as 10, 20, and 5. 5 is the minimum cost path so it is the optimal solution. This is the local optimum, and in this way, we find the local optimum at each stage in order to calculate the global optimal solution.

LECTURE 2

Exact or Approximate Solving: Which one do you need?

Ever wondered how computers make decisions? Or how they can find the best solutions to complex problems? Well, get ready to unlock the secrets of optimization! In this article, we'll delve into two smart approaches: "exact" or "approximate" methods. But don't worry if you're not a tech wizard – we'll keep things simple and easy to grasp!

To make you in context, Imagine you have a treasure hunt with countless possible paths to reach the treasure. How do you know which route is the absolute best? That's where "exact" methods come in – they tirelessly search every nook and cranny to find the one true solution. But sometimes, you might not have the luxury of time for such a thorough search. That's when "approximate" methods come to the rescue – they quickly find pretty good solutions, even if they're not perfect.

So, how do "exact" methods work their magic to find the best answer to a problem? And how do "approximate" methods cleverly strike a balance between speed and accuracy? And should you go for an exact or an approximate solving? Together, we'll lift the curtain on optimization, and you'll soon be equipped with the knowledge to conquer puzzles and problems like a pro! So, tighten your seatbelt as we embark on this optimizing journey – it's going to be exciting! Let's set sail!

Before The "Exact" or "Approximate" debate, What is Optimization?

Optimization is like being a super problem-solver in the world of mathematics and computer science! At its core, optimization aims to find the best possible solution to a problem, whether it's minimizing costs, maximizing profits, or achieving the highest efficiency.

In more technical terms, optimization involves the art of minimizing or maximizing a specific mathematical function called the "objective function". This function represents what we want to achieve or optimize, like finding the shortest route to a destination, maximizing the output of a production line, or minimizing the time it takes to complete a task.

However, optimization isn't just about blindly finding any solution. It's also about playing by the rules! We must consider certain constraints that define what's possible and what's not. These constraints set limits on the variables involved in the objective function, making sure the solutions we find are practical and realistic.

In the digital world, computers use powerful algorithms to navigate through the vast solution space and discover the best answers to complex problems. From engineering and finance to logistics and scheduling, optimization plays a vital role in making smart decisions and finding efficient solutions.

If you want to pursue an academic program in operations research to learn more about optimization, check our list of the top 49 operations research master's programs in the world for 2024.

Exact vs Approximate Solving: The Optimal Solution

In the world of mathematical optimization, one powerful method is "exact solving." As the name suggests, and unlike approximate solving, exact solving aims to find the exact and optimal solution to an optimization problem, leaving no room for ambiguity or approximation.

We can be entirely sure that the solution we find is the absolute best among all possible options. The algorithms used in exact solving checks all possible solution in their exhaustive exploration, ensuring that the identified solution is truly unbeatable.

How Exact Solving Works

In the world of exact solving, various algorithms are available, each tailored to specific types of optimization problems. Some exact solvers excel at linear programming, while others are more adept at handling quadratic form programs or constraint programming..etc.

However and despite their specialization, all exact algorithms share the fundamental concept of searching through the entire solution space. It starts by considering all feasible options within the problem's constraints and evaluates each one to determine its objective function value. The algorithm traverses the solution space until it identifies the solution with the best possible objective function value, ensuring no better solution exists.

Popular Exact Solving Algorithms

Several well-known algorithms are used in exact solving to comb through the solution space systematically. Here is a list of the most common and known exact solvers used in mathematical optimization:

- **Simplex Algorithm: This method efficiently solves linear programming problems by iteratively moving along the edges of the feasible region to find the optimal solution.**
- **Branch and Bound: Used for integer programming and mixed-integer programming, it systematically explores the solution space by breaking the problem into smaller subproblems and pruning certain branches to find the best integer solutions.**
- **Cutting-Plane Methods: Specifically designed for integer programming, it narrows down the solution space by adding valid inequalities to eliminate suboptimal solutions.**
- **Lagrange Multipliers: Employed in quadratic form optimization problems with equality constraints, it helps find critical points of the objective function by introducing additional variables.**
- **Karush-Kuhn-Tucker (KKT) Conditions: Used for quadratic form optimization problems with inequality constraints, it provides necessary conditions for finding optimal solutions.**
- **Trust Region Methods: Applied to nonlinear non-quadratic form optimization problems, it explores the solution space efficiently by iteratively refining a trust region around the current solution.**
- **Backtracking Algorithms: Crucial for constraint programming problems, it systematically searches for feasible solutions by enforcing constraints incrementally and avoiding invalid branches.**

Exact vs Approximate Solving: Finding the Close Enough Solution

Before we dive into the depths of approximate solving, let's first see what "approximate" means.

When we talk about an "approximate" solution, unlike with exact solving, we're looking for an answer that's not perfect but comes close enough. An easy example to illustrate is like when you're trying to guess the number of candies in a jar, you might not get it spot-on, but you want to be as close as possible.

Why Approximate Over Exact Solving?

Why would we settle for an "almost there" answer when we can aim for perfection? Well, the truth is, not all problems have an exact solution that's easy to find. Some problems are like mazes with so many twists and turns that even supercomputers can't navigate through them in a reasonable time. And when in doubt about the problem's toughness, we always have polynomial reductions to come to the rescue.

And that's where approximate solving steps in. Instead of spending ages looking for the one true solution, approximate methods get us pretty close, pretty fast. So, when you're facing a problem that's too big or too complex, and you need answers quickly, you turn to approximate methods.

Heuristic vs. Metaheuristic

Now, let's talk about two rockstars of the approximate solving world: heuristics and metaheuristics. They're like the strategy guides for video games – they don't give you every detail, but they show you the path to success.

Heuristics

Now, let's dive into heuristics in a bit more detail. Imagine you're on a treasure hunt in a big forest, and you have a hunch about where the treasure might be hidden. You take a quick shortcut to that spot, even though you're not 100% sure it's where the treasure is.

In optimization problems, heuristics work just like that. They're fast and clever methods that make educated guesses to find solutions. Think of it like playing hide-and-seek – heuristics help you make the best guess on where to look first. They're like shortcuts that might not get you to the treasure, but they'll surely lead you in the right direction.

The interesting thing about heuristics is that they don't guarantee the absolute best solution. Instead, they give you good solutions quickly. If we were talking about a well-known optimization problem, let's say scheduling classes at a school. The classes have different times and rooms, and you want to find a way to fit them all without any overlaps. A heuristic might quickly try different combinations of classes and rooms, looking for a way that works. It might not find the perfect schedule, but it will likely get pretty close in a short time.

Metaheuristics: Versatile Problem-Solvers

Now, let's delve deeper into metaheuristics. Think of them as the Swiss Army knives of optimization. They're general algorithms that can adapt to various problems and provide approximate solutions. Imagine you have a toolbox that can fix different things around your house – that's what metaheuristics do for optimization problems.

Metaheuristics aren't limited to one specific problem. They're like clever problem-solvers that you can train to tackle different challenges. There are different types of metaheuristics, each with its own approach. For instance, there are "greedy" ones that make decisions based on the current best option. Then you have "local search metaheuristics" that fine-tune a solution step by step, looking for improvements.

There are also "constructive metaheuristics" that build a solution piece by piece, like assembling a puzzle. "Population-based metaheuristics" work with a group of solutions, kind of like a team brainstorming to find the best solution together. And if you're really looking for innovation, you've got "hybrid" metaheuristics that combine the strengths of different approaches.

If you want to know more about heuristic evaluation, refer to our specialized post : "Heuristic Evaluation Guide: Pick the Best Heuristic with Confidence"

Which of the following is an example of exact algorithm?

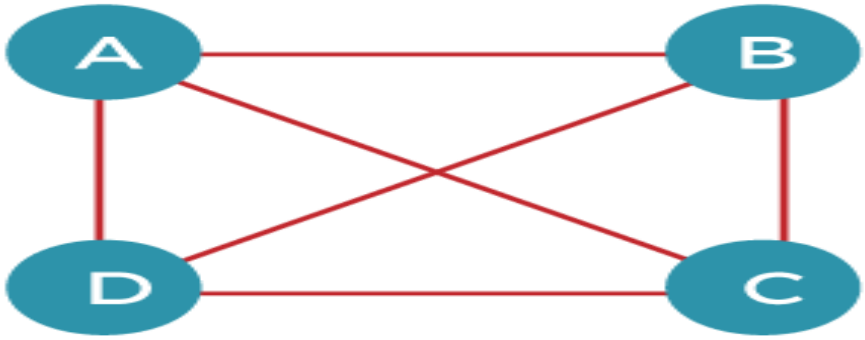
Here are examples: An exact algorithm for the Traveling Salesman problem is NP Complete. The TSP is to find a route of minimum length L for visiting each of N cities on a map. NP Completeness says the best known algorithms still need time that is an exponential function of N

LECTURE 3

Minimum Spanning Tree

Before knowing about the minimum spanning tree, we should know about the spanning tree.

To understand the concept of spanning tree, consider the below graph:



The above graph can be represented as G(V, E), where 'V' is the number of vertices, and 'E' is the number of edges. The spanning tree of the above graph would be represented as G`(V`, E`). In this case, V` = V means that the number of vertices in the spanning tree would be the same as the number of vertices in the graph, but the number of edges would be different. The number of edges in the spanning tree is the subset of the number of edges in the original graph. Therefore, the number of edges can be written as:

E` € E

It can also be written as:

E` = |V| - 1

Two conditions exist in the spanning tree, which is as follows:

- The number of vertices in the spanning tree would be the same as the number of vertices in the original graph.

  V` = V

- The number of edges in the spanning tree would be equal to the number of edges minus 1.

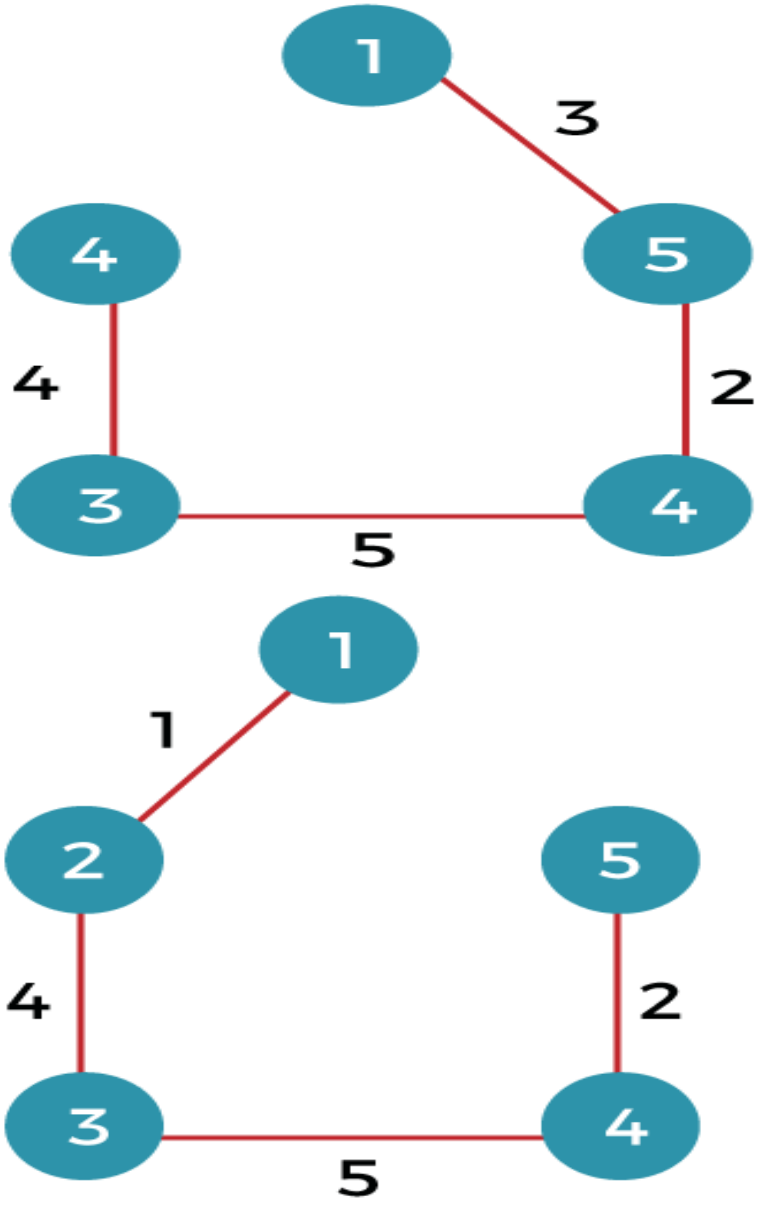
  E` = |V| - 1

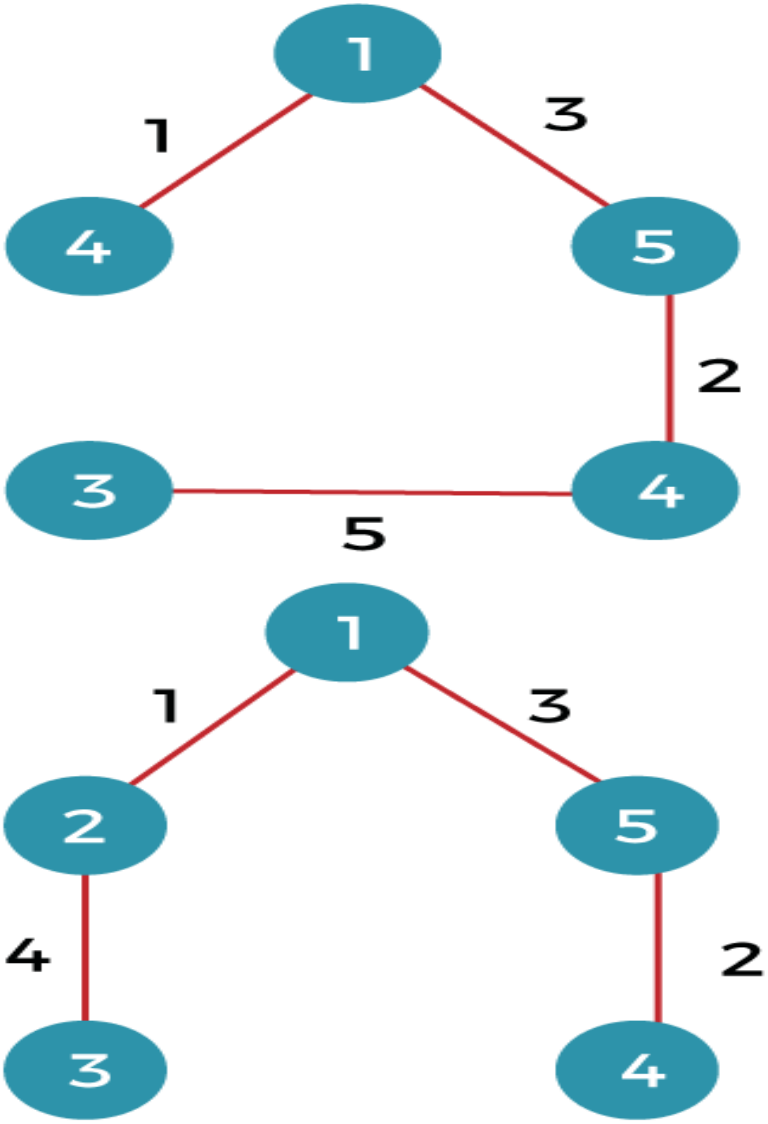
- The spanning tree should not contain any cycle.
- The spanning tree should not be disconnected.

Note: A graph can have more than one spanning tree.

Consider the below graph:

The above graph contains 5 vertices. As we know, the vertices in the spanning tree would be the same as the graph; therefore, V` is equal 5. The number of edges in the spanning tree would be equal to (5 - 1), i.e., 4. The following are the possible spanning trees:





What is a minimum spanning tree?

The minimum spanning tree is a spanning tree whose sum of the edges is minimum. Consider the below graph that contains the edge weight:

The following are the spanning trees that we can make from the above graph.

- The first spanning tree is a tree in which we have removed the edge between the vertices 1 and 5 shown as below:
  The sum of the edges of the above tree is (1 + 4 + 5 + 2): 12
- The second spanning tree is a tree in which we have removed the edge between the vertices 1 and 2 shown as below:
  The sum of the edges of the above tree is (3 + 2 + 5 + 4) : 14

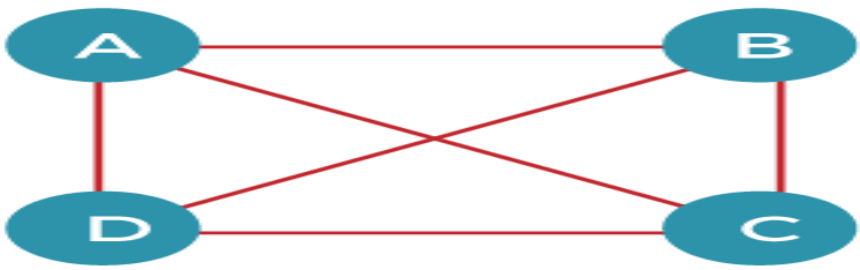
- The third spanning tree is a tree in which we have removed the edge between the vertices 2 and 3 shown as below:
  The sum of the edges of the above tree is (1 + 3 + 2 + 5) : 11
- The fourth spanning tree is a tree in which we have removed the edge between the vertices 3 and 4 shown as below:
  The sum of the edges of the above tree is (1 + 3 + 2 + 4) : 10. The edge cost 10 is minimum so it is a minimum spanning tree.

General properties of minimum spanning tree:

- If we remove any edge from the spanning tree, then it becomes disconnected. Therefore, we cannot remove any edge from the spanning tree.
- If we add an edge to the spanning tree then it creates a loop. Therefore, we cannot add any edge to the spanning tree.
- In a graph, each edge has a distinct weight, then there exists only a single and unique minimum spanning tree. If the edge weight is not distinct, then there can be more than one minimum spanning tree.
- A complete undirected graph can have an n^{n-2} number of spanning trees.
- Every connected and undirected graph contains atleast one spanning tree.
- The disconnected graph does not have any spanning tree.
- In a complete graph, we can remove maximum (e-n+1) edges to construct a spanning tree.

Let's understand the last property through an example.

Consider the complete graph which is given below:



The number of spanning trees that can be made from the above complete graph equals to $n^{n-2} = 4^{4-2} = 16$.

Therefore, 16 spanning trees can be created from the above graph.

The maximum number of edges that can be removed to construct a spanning tree equals to e-n+1 = 6 - 4 + 1 = 3.

LECTURE 4

Introduction to Knapsack Problem, its Types and How to solve them

The **Knapsack problem** is an example of the combinational optimization problem. This problem is also commonly known as the "**Rucksack Problem**". The name of the problem is defined from the maximization problem as mentioned below:
Given a bag with maximum weight capacity of W and a set of items, each having a weight and a value associated with it. Decide the number of each item to take in a collection such that the total weight is less than the capacity and the total value is maximized.

Types of Knapsack Problem:
The knapsack problem can be classified into the following types:

1. Fractional Knapsack Problem
2. 0/1 Knapsack Problem
3. Bounded Knapsack Problem
4. Unbounded Knapsack Problem

1. Fractional Knapsack Problem

The Fractional Knapsack problem can be defined as follows:

Given the weights and values of **N** items, put these items in a knapsack of capacity **W** to get the maximum total value in the knapsack. In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

2. 0/1 Knapsack Problem

The 0/1 Knapsack problem can be defined as follows:

We are given **N** items where each item has some weight (w_i) and value (v_i) associated with it. We are also given a bag with capacity **W**. The target is to put the items into the bag such that the sum of values associated with them is the maximum possible.
Note that here we can either put an item completely into the bag or cannot put it at all.

Mathematically the problem can be expressed as:

Maximize subject to and $x_i \in \{0, 1\}$

Following are the differences between the 0/1 knapsack problem and the Fractional knapsack problem**.**

Sr. No	**0/1 knapsack problem**	**Fractional knapsack problem**
1.	The 0/1 knapsack problem is solved using dynamic programming approach.	Fractional knapsack problem is solved using a greedy approach.
2.	The 0/1 knapsack problem has not an	The fractional knapsack problem has an

Sr. No	0/1 knapsack problem	Fractional knapsack problem
	optimal structure.	optimal structure.
3.	In the 0/1 knapsack problem, we are not allowed to break items.	Fractional knapsack problem, we can break items for maximizing the total value of the knapsack.
4.	0/1 knapsack problem, finds a most valuable subset item with a total value less than equal to weight.	In the fractional knapsack problem, finds a most valuable subset item with a total value equal to the weight.
5.	In the 0/1 knapsack problem we can take objects in an integer value.	In the fractional knapsack problem, we can take objects in fractions in floating points.

3. Bounded Knapsack Problem

The Bounded Knapsack problem can be defined as follows:

Given **N** items, each item having a given weight $\mathbf{w_i}$ and a value $\mathbf{v_i}$, the task is to maximize the value by selecting a maximum of **K** items adding up to a maximum weight **W**.
Mathematically the problem can be expressed as:

Maximize subject to and $x_i \in \{0, 1, \ldots, K\}$

4. Unbounded Knapsack Problem

The Unbounded Knapsack problem can be defined as follows:

Given a knapsack weight **W** and a set of **N** items with certain value $\mathbf{v_i}$ and weight $\mathbf{w_i}$, we need to calculate the maximum amount that could make up this quantity exactly. This is different from **0/1 Knapsack problem**, here we are allowed to use an unlimited number of instances of an item.
Mathematically the problem can be expressed as:

Maximize subject to and and $x_i \geq 0$.

Variations of Knapsack Problem:

There are several variations possible for the Knapsack Problem. Some of the well-known variations are provided below:

1. Multi-objective Knapsack problem:

In this variation, the goal of filling the knapsack changes. Instead of maximizing only the value, there can be several other objectives.

For example: Consider you are organizing a music show in a hall that has a capacity of 10,000. You are organizing a show and the size of the audience depends on the popularity of the singers. Also, the more popular the singer is, the more the fee. You want to maximize the profit and minimize the amount spend on the singer simultaneously and also want to bring as many singers as possible.

2. Multi-dimensional Knapsack problem:

In this variation of the problem, the weight of any item **i** is given by an M dimensional vector $\{w_{i1}, w_{i2}, \ldots w_{iM}\}$ and similarly, the capacity of the knapsack is also an M dimensional vector $\{W_1, W_2, \ldots, W_M\}$.

3. Multiple Knapsack problem:

This variation of the knapsack problem is similar to the **Bin packing algorithm**. The difference in both the problem is here we can pick a subset of the items whereas, in the Bin Packing problem, we have to pack all the items in any of the bins. The idea is that there are multiple knapsacks which may seem like adding capacity to the initial knapsack, but it is not similar to that at all.

- Double Knapsack

4. Quadratic Knapsack problem:

This variation has the goal of achieving the maximum value of a quadratic objective function that is subjected to binary and linear capacity constraints.

5. Geometric Knapsack problem:

In this variation, there is a set of rectangles with different values and a rectangular knapsack. The goal is to pack the largest possible value into the knapsack.

Applications of the Knapsack Problem:

The Knapsack problem has several real-life applications. Some of them are mentioned here:

- One of the early applications of the Knapsack problem was in construction and scoring of exams in which the test takers have a choice as to which questions they answer.
- The subset sum problem is solved using the concept of the Knapsack problem.
- The multiple objective variations of the Knapsack problem is frequently used for transportation logistics optimization problems.
- The multiple knapsack problem is often used in many loading and scheduling algorithms in Operational Research.

KRUSKAL ALGORITHM

The Kruskal Algorithm is used to find the minimum cost of a spanning tree. A spanning tree is a connected graph using all the vertices in which there are no loops. In other words, we can say that there is a path from any vertex to any other vertex but no loops.

What is Minimum Cost Spanning Tree?

The minimum spanning tree is a spanning tree that has the smallest total edge weight. The Kruskal algorithm is an algorithm that takes the graph as input and finds the edges from the graph, which forms a tree that includes every vertex of a graph.

Working of Kruskal Algorithm

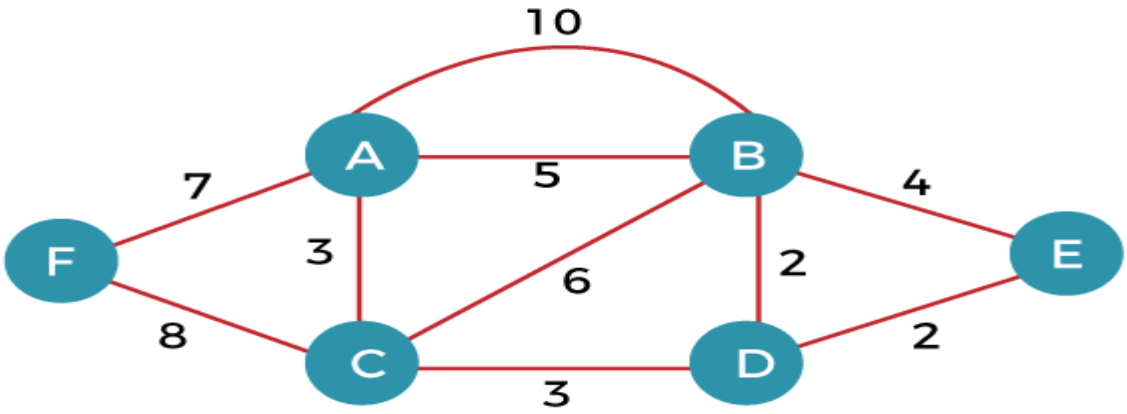
The working of the Kruskal algorithm starts from the edges, which has the lowest weight and keeps adding the edges until we reach the goal.

The following are the steps used to implement the Kruskal algorithm:

- First, sort the edges in the ascending order of their edge weights.
- Consider the edge which is having the lowest weight and add it in the spanning tree. If adding any edge in a spanning tree creates a cycle then reject that edge.
- Keep adding the edges until we reach the end vertex.

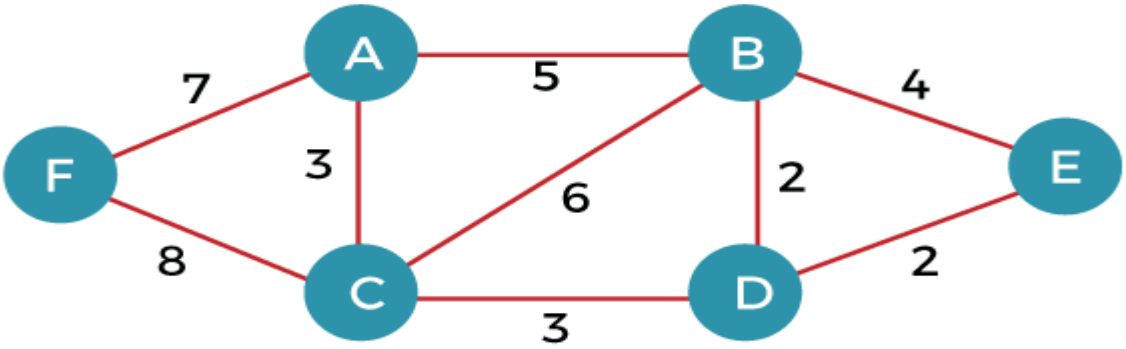
Let's understand through an example.

Consider the below graph.



Now we have to calculate the minimum spanning tree of the above graph. To find the minimum cost spanning tree, the first step is to remove all the loops and the parallel edges from the graph. Here, parallel edges mean that there exists more than one edge between the two vertices. For example, in the above graph, there exists two edges between the vertices A and B having weights 5 and 10 respectively, so these edges are the parallel edges. We have to remove any of the edges.

In case of parallel edges, the edges with the highest weight will be removed. In the above graph, if we consider the vertices A and B then the edge with a weight 10 is more than the edge with weight 5 so we discard the edge having weight 10. So, we remove the edge having weight 10 as shown as below:



Once we remove the parallel edge, we will arrange the edges according to the increasing order of their edge weights. As we can observe in the above graph, the minimum edge is 2. There are two edges having weight equals to 2 which can be written as:

BD = 2

DE = 2

The next minimum edge weight is 3. There are two edges with a weight 3 so it can be written as:

AC = 3

CD = 3

The next edge with a minimum weight is 4. There is only one edge having weight 4 and it can be written as:

BE = 4

The next edge with a minimum weight is 5. There is only one edge having weight 5 and it can be written as:

AB = 5

The next edge with a minimum weight is 6. There is only one edge having weight 6 and it can be written as:

CB = 6

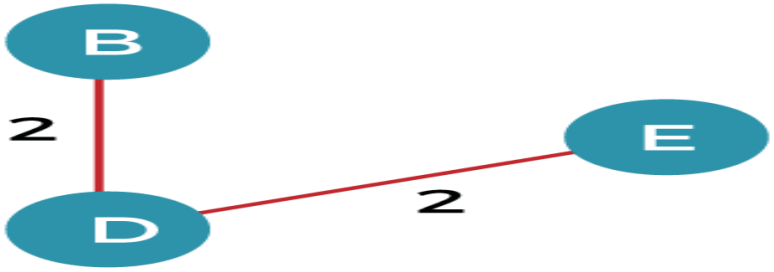
The next edge with a minimum weight is 7. There is only one edge having weight 7 and it can be written as:

AF = 6

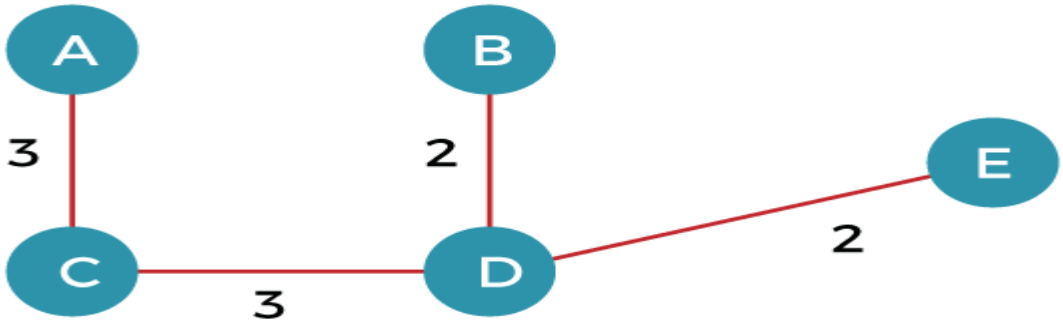
The next edge with a minimum weight is 8. There is only one edge having weight 8 and it can be written as:

FC = 6

Once the edge weights are written in the increasing order, the third step is to connect the vertices according to their weights. The edge weight 2 is minimum, so we connect the vertices with a edge weight 2 as shown as below:



The next edge is AC having weight 3 shown as below:



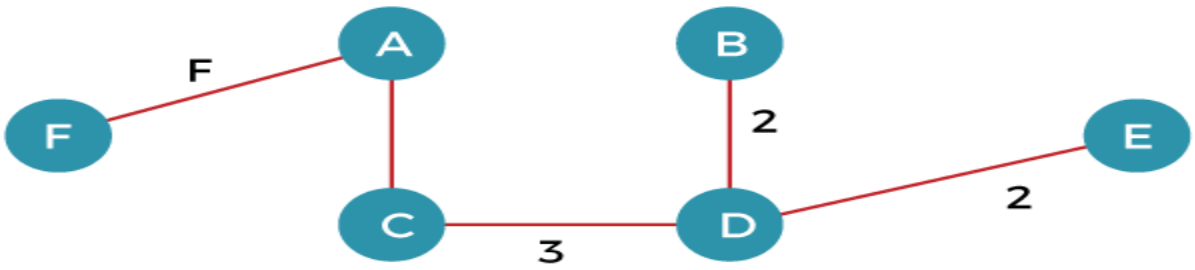
The next edge is CD. If we connect the vertices C and D then it does not form any cycle shown as below:

The next edge is BE. But we cannot connect the vertices B and E as it forms a cycle.

The next edge is AB. But we cannot connect the vertices A and B as it forms a cycle.

The next edge is BC. But we cannot connect the vertices B and C as it forms a cycle.

The next edge is AF. If we connect the vertices A and F then it does not form any cycle shown as below:



Algorithm

1. Step 1: Create a forest F in such a way that every vertex of the graph is a separate tree.
2. Step 2: Create a set E that contains all the edges of the graph.
3. Step 3: Repeat Steps 4 and 5 while E is NOT EMPTY and F is not spanning
4. Step 4: Remove an edge from E with minimum weight
5. Step 5: IF the edge obtained in Step 4 connects two different trees, then add it to the forest F

6. (for combining two trees into one tree).
7. ELSE
8. Discard the edge
9. Step 6: END

LECTURE 5

PRIM'S ALGORITHM

In this article, we will discuss the prim's algorithm. Along with the algorithm, we will also see the complexity, working, example, and implementation of prim's algorithm.

Before starting the main topic, we should discuss the basic and important terms such as spanning tree and minimum spanning tree.

Spanning tree - A spanning tree is the subgraph of an undirected connected graph.

Minimum Spanning tree - Minimum spanning tree can be defined as the spanning tree in which the sum of the weights of the edge is minimum. The weight of the spanning tree is the sum of the weights given to the edges of the spanning tree.

Now, let's start the main topic.

Prim's Algorithm is a greedy algorithm that is used to find the minimum spanning tree from a graph. Prim's algorithm finds the subset of edges that includes every vertex of the graph such that the sum of the weights of the edges can be minimized.

Prim's algorithm starts with the single node and explores all the adjacent nodes with all the connecting edges at every step. The edges with the minimal weights causing no cycles in the graph got selected.

How does the prim's algorithm work?

Prim's algorithm is a greedy algorithm that starts from one vertex and continue to add the edges with the smallest weight until the goal is reached. The steps to implement the prim's algorithm are given as follows -

- First, we have to initialize an MST with the randomly chosen vertex.
- Now, we have to find all the edges that connect the tree in the above step with the new vertices. From the edges found, select the minimum edge and add it to the tree.
- Repeat step 2 until the minimum spanning tree is formed.

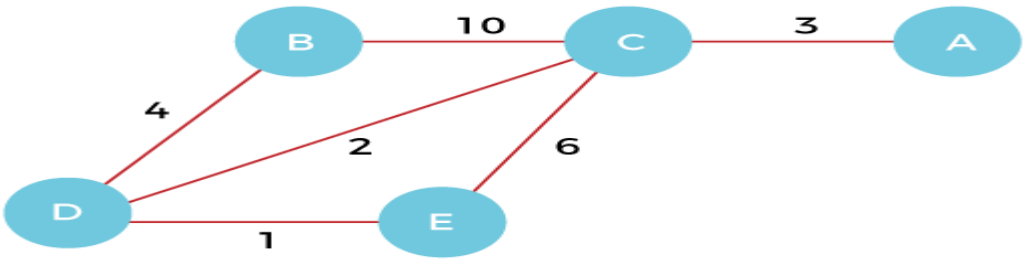
The applications of prim's algorithm are -

- Prim's algorithm can be used in network designing.
- It can be used to make network cycles.
- It can also be used to lay down electrical wiring cables.

Example of prim's algorithm

Now, let's see the working of prim's algorithm using an example. It will be easier to understand the prim's algorithm using an example.

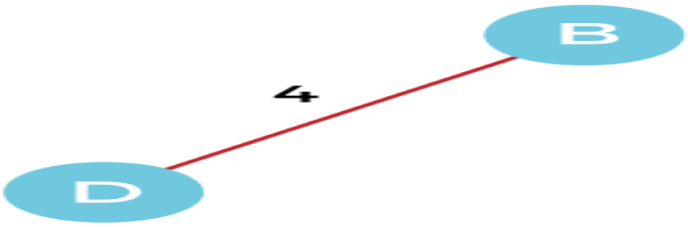
Suppose, a weighted graph is -



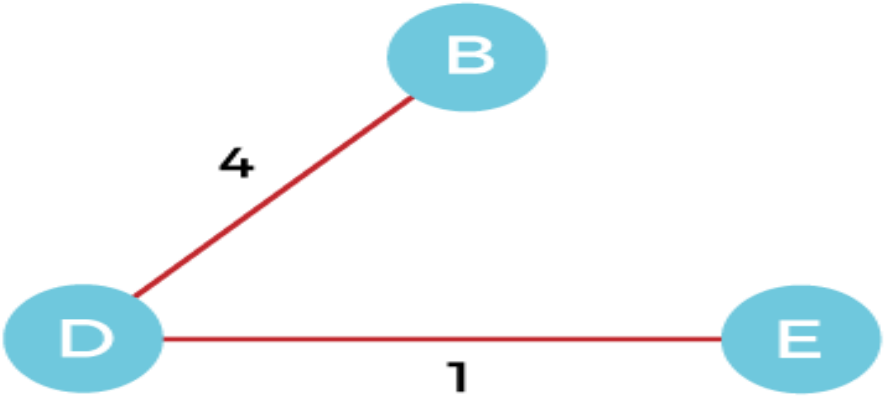
Step 1 - First, we have to choose a vertex from the above graph. Let's choose B.



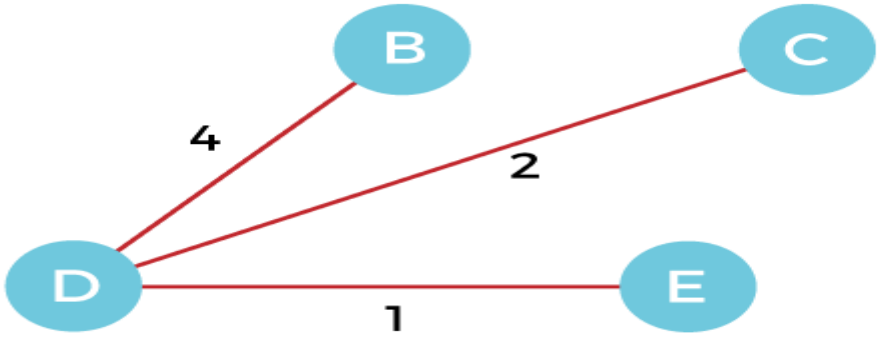
Step 2 - Now, we have to choose and add the shortest edge from vertex B. There are two edges from vertex B that are B to C with weight 10 and edge B to D with weight 4. Among the edges, the edge BD has the minimum weight. So, add it to the MST.



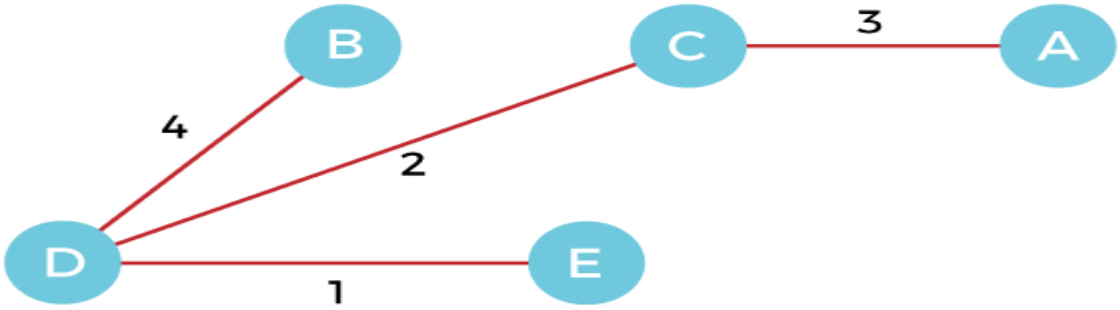
Step 3 - Now, again, choose the edge with the minimum weight among all the other edges. In this case, the edges DE and CD are such edges. Add them to MST and explore the adjacent of C, i.e., E and A. So, select the edge DE and add it to the MST.



Step 4 - Now, select the edge CD, and add it to the MST.



Step 5 - Now, choose the edge CA. Here, we cannot select the edge CE as it would create a cycle to the graph. So, choose the edge CA and add it to the MST.



So, the graph produced in step 5 is the minimum spanning tree of the given graph. The cost of the MST is given below -

Cost of MST = 4 + 2 + 1 + 3 = 10 units.

Algorithm

1. Step 1: Select a starting vertex
2. Step 2: Repeat Steps 3 and 4 until there are fringe vertices
3. Step 3: Select an edge 'e' connecting the tree vertex and fringe vertex that has minimum weig ht
4. Step 4: Add the selected edge and the vertex to the minimum spanning tree T
5. [END OF LOOP]
6. Step 5: EXIT

Complexity of Prim's algorithm

Now, let's see the time complexity of Prim's algorithm. The running time of the prim's algorithm depends upon using the data structure for the graph and the ordering of edges. Below table shows some choices -

Data structure used for the minimum edge weight	Time Complexity
Adjacency matrix, linear searching	$O(\|V\|^2)$
Adjacency list and binary heap	$O(\|E\| \log \|V\|)$
Adjacency list and Fibonacci heap	$O(\|E\|+ \|V\| \log \|V\|)$

- **Time Complexity**

Prim's algorithm can be simply implemented by using the adjacency matrix or adjacency list graph representation, and to add the edge with the minimum weight requires the linearly searching of an array of weights. It requires $O(|V|^2)$ running time. It can be improved further by using the implementation of heap to find the minimum weight edges in the inner loop of the algorithm.

The time complexity of the prim's algorithm is O(E logV) or O(V logV), where E is the no. of edges, and V is the no. of vertices.

Implementation of Prim's algorithm

Now, let's see the implementation of prim's algorithm.

LECTURE 6

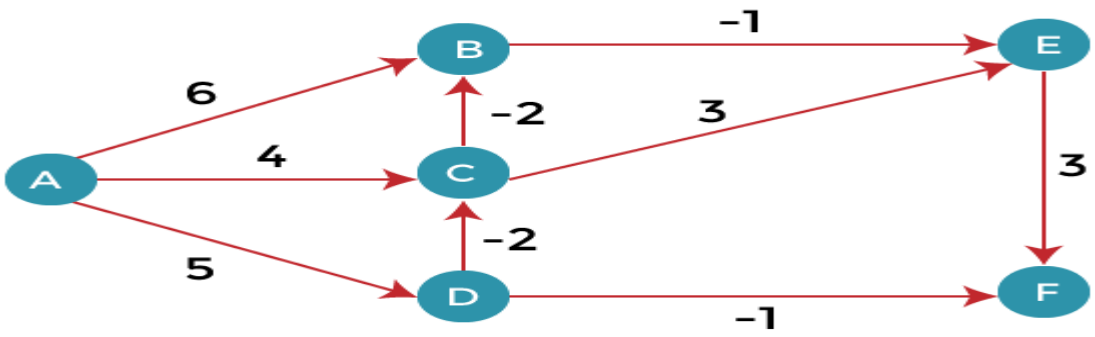
Bellman Ford Algorithm

Bellman ford algorithm is a single-source shortest path algorithm. This algorithm is used to find the shortest distance from the single vertex to all the other vertices of a weighted graph. There are various other algorithms used to find the shortest path like Dijkstra algorithm, etc. If the weighted graph contains the negative weight values, then the Dijkstra algorithm does not confirm whether it produces the correct answer or not. In contrast to Dijkstra algorithm, bellman ford algorithm guarantees the correct answer even if the weighted graph contains the negative weight values.

Rule of this algorithm

1. We will go on relaxing all the edges (n - 1) times where,
2. n = number of vertices

Consider the below graph:



As we can observe in the above graph that some of the weights are negative. The above graph contains 6 vertices so we will go on relaxing till the 5 vertices. Here, we will relax all the edges 5 times. The loop will iterate 5 times to get the correct answer. If the loop is iterated more than 5 times then also the answer will be the same, i.e., there would be no change in the distance between the vertices.

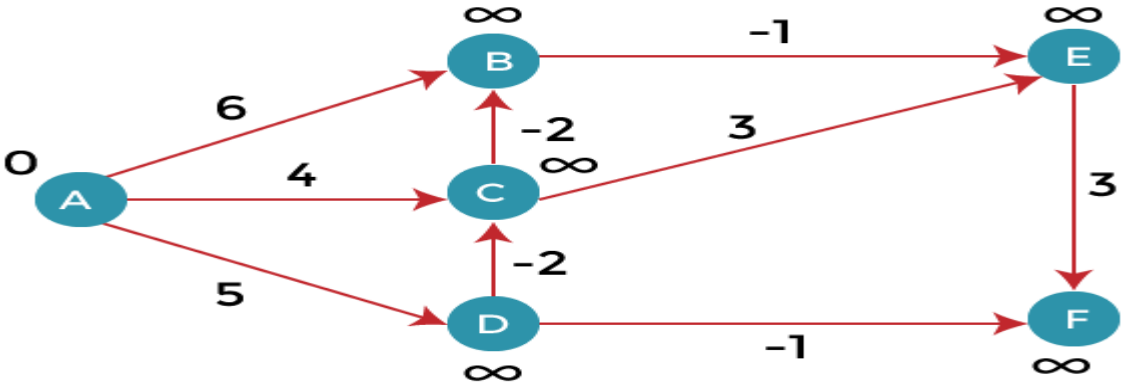
Relaxing means:

1. If (d(u) + c(u , v) < **d**(v))
2. d(v) = d(u) + c(u , v)

To find the shortest path of the above graph, the first step is note down all the edges which are given below:

(A, B), (A, C), (A, D), (B, E), (C, E), (D, C), (D, F), (E, F), (C, B)

Let's consider the source vertex as 'A'; therefore, the distance value at vertex A is 0 and the distance value at all the other vertices as infinity shown as below:



Since the graph has six vertices so it will have five iterations.

First iteration

Consider the edge (A, B). Denote vertex 'A' as 'u' and vertex 'B' as 'v'. Now use the relaxing formula:

d(u) = 0

d(v) = ∞

c(u , v) = 6

Since (0 + 6) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 0 + 6 = 6

Therefore, the distance of vertex B is 6.

Consider the edge (A, C). Denote vertex 'A' as 'u' and vertex 'C' as 'v'. Now use the relaxing formula:

d(u) = 0

d(v) = ∞

c(u , v) = 4

Since (0 + 4) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 0 + 4 = 4

Therefore, the distance of vertex C is 4.

Consider the edge (A, D). Denote vertex 'A' as 'u' and vertex 'D' as 'v'. Now use the relaxing formula:

d(u) = 0

d(v) = ∞

c(u , v) = 5

Since (0 + 5) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 0 + 5 = 5

Therefore, the distance of vertex D is 5.

Consider the edge (B, E). Denote vertex 'B' as 'u' and vertex 'E' as 'v'. Now use the relaxing formula:

d(u) = 6

d(v) = ∞

c(u , v) = -1

Since (6 - 1) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 6 - 1= 5

Therefore, the distance of vertex E is 5.

Consider the edge (C, E). Denote vertex 'C' as 'u' and vertex 'E' as 'v'. Now use the relaxing formula:

d(u) = 4

d(v) = 5

c(u , v) = 3

Since (4 + 3) is greater than 5, so there will be no updation. The value at vertex E is 5.

Consider the edge (D, C). Denote vertex 'D' as 'u' and vertex 'C' as 'v'. Now use the relaxing formula:

d(u) = 5

d(v) = 4

c(u , v) = -2

Since (5 -2) is less than 4, so update

1. d(v) = d(u) + c(u , v)

d(v) = 5 - 2 = 3

Therefore, the distance of vertex C is 3.

Consider the edge (D, F). Denote vertex 'D' as 'u' and vertex 'F' as 'v'. Now use the relaxing formula:

d(u) = 5

d(v) = ∞

c(u , v) = -1

Since (5 -1) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 5 - 1 = 4

Therefore, the distance of vertex F is 4.

Consider the edge (E, F). Denote vertex 'E' as 'u' and vertex 'F' as 'v'. Now use the relaxing formula:

d(u) = 5

d(v) = ∞

c(u , v) = 3

Since (5 + 3) is greater than 4, so there would be no updation on the distance value of vertex F.

Consider the edge (C, B). Denote vertex 'C' as 'u' and vertex 'B' as 'v'. Now use the relaxing formula:

d(u) = 3

d(v) = 6

c(u , v) = -2

Since (3 - 2) is less than 6, so update

1. d(v) = d(u) + c(u , v)

d(v) = 3 - 2 = 1

Therefore, the distance of vertex B is 1.

Now the first iteration is completed. We move to the second iteration.

Second iteration:

In the second iteration, we again check all the edges. The first edge is (A, B). Since (0 + 6) is greater than 1 so there would be no updation in the vertex B.

The next edge is (A, C). Since (0 + 4) is greater than 3 so there would be no updation in the vertex C.

The next edge is (A, D). Since (0 + 5) equals to 5 so there would be no updation in the vertex D.

The next edge is (B, E). Since (1 - 1) equals to 0 which is less than 5 so update:

d(v) = d(u) + c(u, v)

d(E) = d(B) +c(B , E)

= 1 - 1 = 0

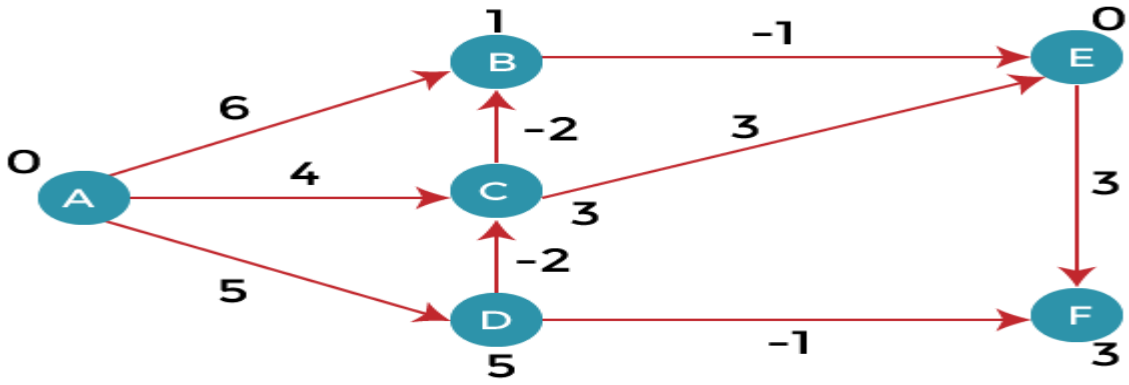
The next edge is (C, E). Since (3 + 3) equals to 6 which is greater than 5 so there would be no updation in the vertex E.

The next edge is (D, C). Since (5 - 2) equals to 3 so there would be no updation in the vertex C.

The next edge is (D, F). Since (5 - 1) equals to 4 so there would be no updation in the vertex F.

The next edge is (E, F). Since (5 + 3) equals to 8 which is greater than 4 so there would be no updation in the vertex F.

The next edge is (C, B). Since (3 - 2) equals to 1` so there would be no updation in the vertex B.



Third iteration

We will perform the same steps as we did in the previous iterations. We will observe that there will be no updation in the distance of vertices.

1. The following are the distances of vertices:
2. A: 0
3. B: 1
4. C: 3
5. D: 5
6. E: 0
7. F: 3

Time Complexity

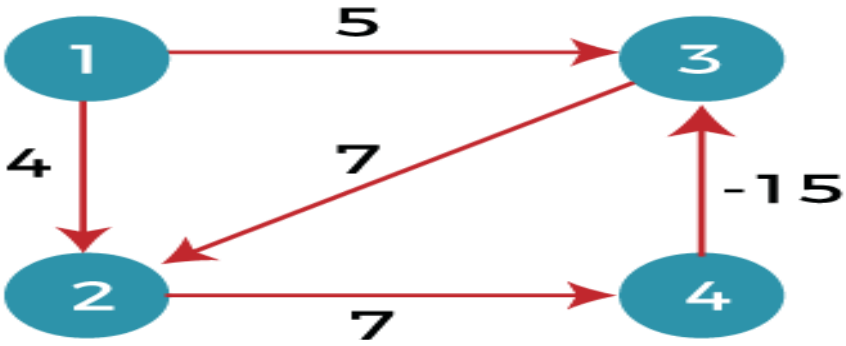
The time complexity of Bellman ford algorithm would be O(E|V| - 1).

```
1.  function bellmanFord(G, S)
2.    for each vertex V in G
3.      distance[V] <- infinite
4.        previous[V] <- NULL
5.    distance[S] <- 0
6.
7.    for each vertex V in G
8.      for each edge (U,V) in G
9.        tempDistance <- distance[U] + edge_weight(U, V)
10.       if tempDistance < distance[V]
11.         distance[V] <- tempDistance
```

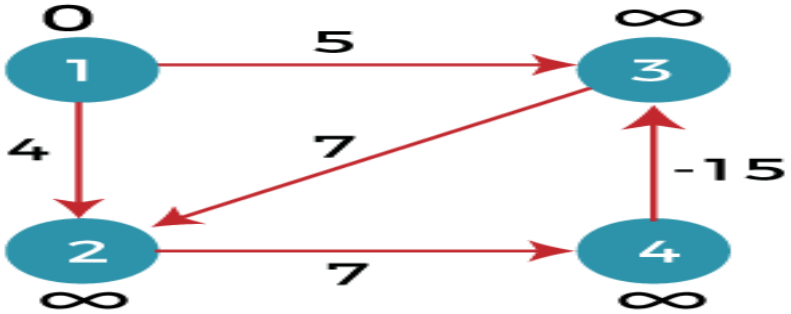
```
12.        previous[V] <- U
13.
14.   for each edge (U,V) in G
15.    If distance[U] + edge_weight(U, V) < distance[V}
16.      Error: Negative Cycle Exists
17.
18.   return distance[], previous[]
```

Drawbacks of Bellman ford algorithm

- The bellman ford algorithm does not produce a correct answer if the sum of the edges of a cycle is negative. Let's understand this property through an example. Consider the below graph.



- In the above graph, we consider vertex 1 as the source vertex and provides 0 value to it. We provide infinity value to other vertices shown as below:



Edges can be written as:

(1, 3), (1, 2), (2, 4), (3, 2), (4, 3)

First iteration

Consider the edge (1, 3). Denote vertex '1' as 'u' and vertex '3' as 'v'. Now use the relaxing formula:

d(u) = 0

d(v) = ∞

c(u , v) = 5

Since (0 + 5) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 0 + 5 = 5

Therefore, the distance of vertex 3 is 5.

Consider the edge (1, 2). Denote vertex '1' as 'u' and vertex '2' as 'v'. Now use the relaxing formula:

d(u) = 0

d(v) = ∞

c(u , v) = 4

Since (0 + 4) is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 0 + 4 = 4

Therefore, the distance of vertex 2 is 4.

Consider the edge (3, 2). Denote vertex '3' as 'u' and vertex '2' as 'v'. Now use the relaxing formula:

d(u) = 5

d(v) = 4

c(u , v) = 7

Since (5 + 7) is greater than 4, so there would be no updation in the vertex 2.

Consider the edge (2, 4). Denote vertex '2' as 'u' and vertex '4' as 'v'. Now use the relaxing formula:

d(u) = 4

d(v) = ∞

c(u , v) = 7

Since (4 + 7) equals to 11 which is less than ∞, so update

1. d(v) = d(u) + c(u , v)

d(v) = 4 + 7 = 11

Therefore, the distance of vertex 4 is 11.

Consider the edge (4, 3). Denote vertex '4' as 'u' and vertex '3' as 'v'. Now use the relaxing formula:

d(u) = 11

d(v) = 5

c(u , v) = -15

Since (11 - 15) equals to -4 which is less than 5, so update

1. d(v) = d(u) + c(u , v)

d(v) = 11 - 15 = -4

Therefore, the distance of vertex 3 is -4.

Now we move to the second iteration.

Second iteration

Now, again we will check all the edges. The first edge is (1, 3). Since (0 + 5) equals to 5 which is greater than -4 so there would be no updation in the vertex 3.

The next edge is (1, 2). Since (0 + 4) equals to 4 so there would be no updation in the vertex 2.

The next edge is (3, 2). Since (-4 + 7) equals to 3 which is less than 4 so update:

d(v) = d(u) + c(u, v)

d(2) = d(3) +c(3, 2)

= -4 + 7 = 3

Therefore, the value at vertex 2 is 3.

The next edge is (2, 4). Since (3+7) equals to 10 which is less than 11 so update

d(v) = d(u) + c(u, v)

d(4) = d(2) +c(2, 4)

= 3 + 7 = 10

Therefore, the value at vertex 4 is 10.

The next edge is (4, 3). Since (10 - 15) equals to -5 which is less than -4 so update:

d(v) = d(u) + c(u, v)

d(3) = d(4) +c(4, 3)

= 10 - 15 = -5

Therefore, the value at vertex 3 is -5.

Now we move to the third iteration.

Third iteration

Now again we will check all the edges. The first edge is (1, 3). Since (0 + 5) equals to 5 which is greater than -5 so there would be no updation in the vertex 3.

The next edge is (1, 2). Since (0 + 4) equals to 4 which is greater than 3 so there would be no updation in the vertex 2.

The next edge is (3, 2). Since (-5 + 7) equals to 2 which is less than 3 so update:

d(v) = d(u) + c(u, v)

d(2) = d(3) +c(3, 2)

= -5 + 7 = 2

Therefore, the value at vertex 2 is 2.

The next edge is (2, 4). Since (2 + 7) equals to 9 which is less than 10 so update:

d(v) = d(u) + c(u, v)

d(4) = d(2) +c(2, 4)

= 2 + 7 = 9

Therefore, the value at vertex 4 is 9.

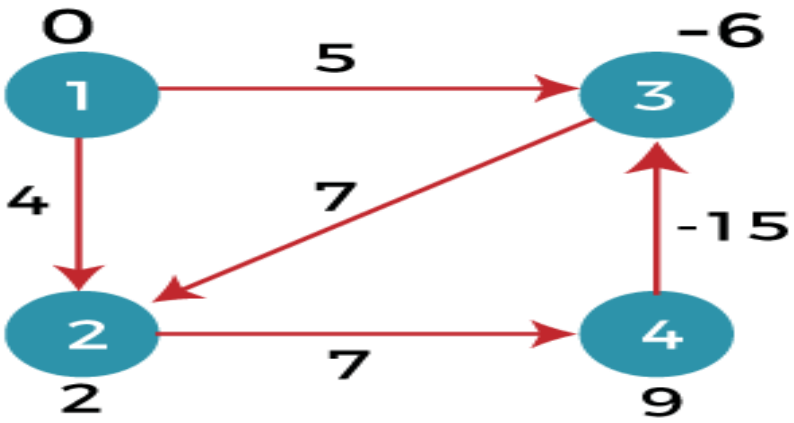
The next edge is (4, 3). Since (9 - 15) equals to -6 which is less than -5 so update:

d(v) = d(u) + c(u, v)

d(3) = d(4) +c(4, 3)

= 9 - 15 = -6

Therefore, the value at vertex 3 is -6.



Since the graph contains 4 vertices, so according to the bellman ford algorithm, there would be only 3 iterations. If we try to perform 4th iteration on the graph, the distance of the vertices from the given vertex should not change. If the distance varies, it means that the bellman ford algorithm is not providing the correct answer.

4th iteration

The first edge is (1, 3). Since (0 +5) equals to 5 which is greater than -6 so there would be no change in the vertex 3.

The next edge is (1, 2). Since (0 + 4) is greater than 2 so there would be no updation.

The next edge is (3, 2). Since (-6 + 7) equals to 1 which is less than 3 so update:

d(v) = d(u) + c(u, v)

d(2) = d(3) +c(3, 2)

= -6 + 7 = 1

In this case, the value of the vertex is updated. So, we conclude that the bellman ford algorithm does not work when the graph contains the negative weight cycle.

Therefore, the value at vertex 2 is 1.

Dijkstra's Algorithm

The following tutorial will teach us about Dijkstra's Shortest Path Algorithm. We will understand the working of Dijkstra's Algorithm with a stepwise graphical explanation.

We will cover the following:

- A Brief Overview of the Fundamental Concepts of Graph
- Understand the Use of Dijkstra's Algorithm
- Understand the Working of the Algorithm with a Step-by-Step Example

So, let's get started.

A Brief Introduction to Graphs

Graphs are non-linear data structures representing the "connections" between the elements. These elements are known as the **Vertices**, and the lines or arcs that connect any two vertices in the graph are known as the **Edges**. More formally, a Graph comprises **a set of Vertices (V)** and **a set of Edges (E)**. The Graph is denoted by **G(V, E)**.

Components of a Graph

1. **Vertices:** Vertices are the basic units of the graph used to represent real-life objects, persons, or entities. Sometimes, vertices are also known as Nodes.
2. **Edges:** Edges are drawn or used to connect two vertices of the graph. Sometimes, edges are also known as Arcs.

The following figure shows a graphical representation of a Graph:

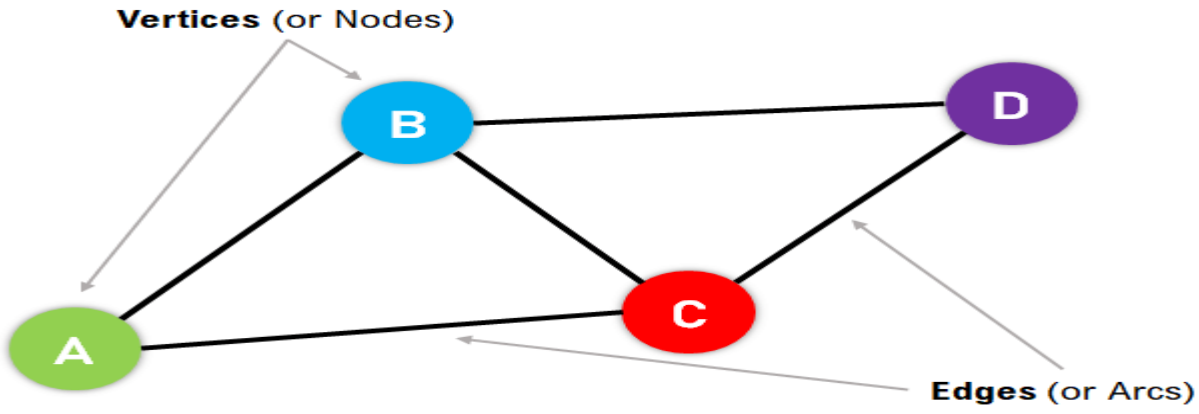


Figure 1: Graphical Representation of a Graph

In the above figure, the Vertices/Nodes are denoted with Colored Circles, and the Edges are denoted with the lines connecting the nodes.

Applications of the Graphs

Graphs are used to solve many real-life problems. Graphs are utilized to represent the networks. These networks may include telephone or circuit networks or paths in a city.

For example, we could use Graphs to design a transportation network model where the vertices display the facilities that send or receive the products, and the edges represent roads or paths connecting them. The following is a pictorial representation of the same:

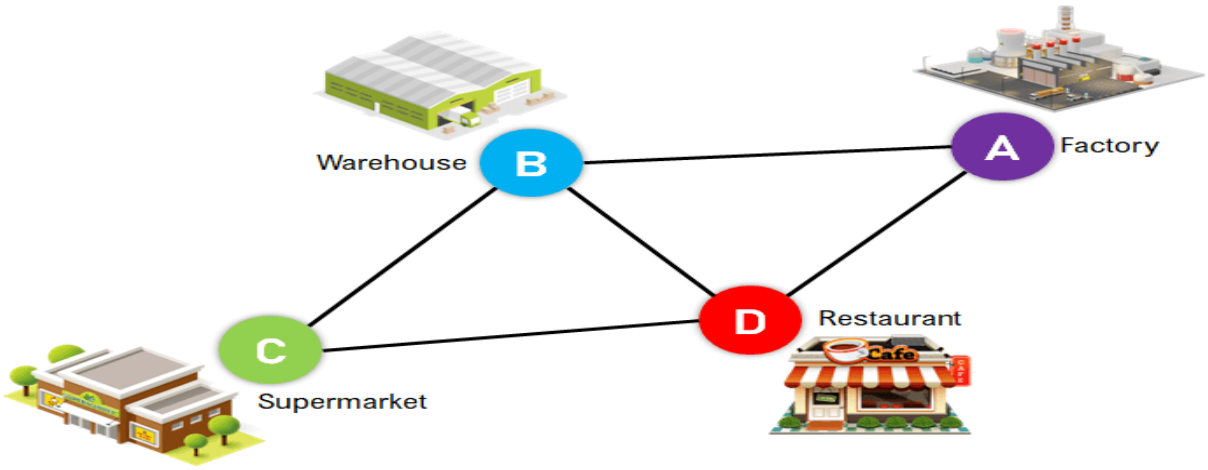


Figure 2: Pictorial Representation of Transportation Network

Graphs are also utilized in different Social Media Platforms like LinkedIn, Facebook, Twitter, and more. For example, Platforms like Facebook use Graphs to store the data of their users where every person is indicated with a vertex, and each of them is a structure containing information like Person ID, Name, Gender, Address, etc.

Types of Graphs

The Graphs can be categorized into two types:

1. Undirected Graph
2. Directed Graph

Undirected Graph: A Graph with edges that do not have a direction is termed an Undirected Graph. The edges of this graph imply a two-way relationship in which each edge can be traversed in both directions. The following figure displays a simple undirected graph with four nodes and five edges.

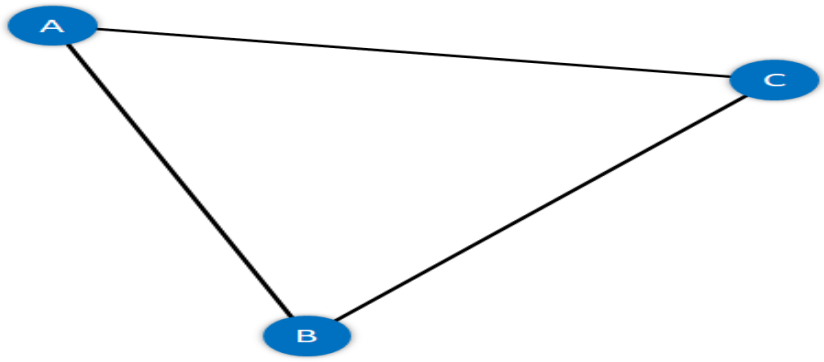


Figure 3: A Simple Undirected Graph

Directed Graph: A Graph with edges with direction is termed a Directed Graph. The edges of this graph imply a one-way relationship in which each edge can only be traversed in a single direction. The following figure displays a simple directed graph with four nodes and five edges.

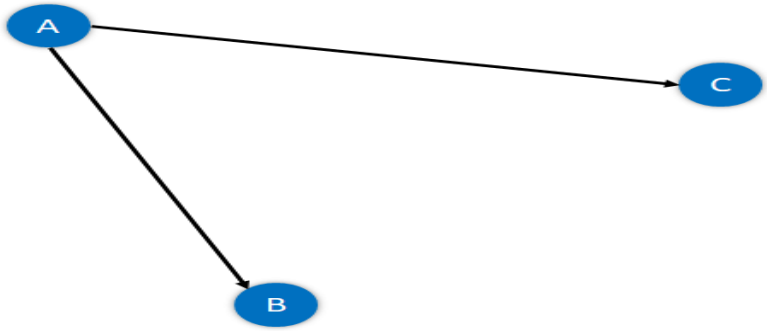


Figure 4: A Simple Directed Graph

The absolute length, position, or orientation of the edges in a graph illustration characteristically does not have meaning. In other words, we can visualize the same graph in different ways by rearranging the vertices or distorting the edges if the underlying structure of the graph does not alter.

What are Weighted Graphs?

A Graph is said to be Weighted if each edge is assigned a 'weight'. The weight of an edge can denote distance, time, or anything that models the 'connection' between the pair of vertices it connects.

For instance, we can observe a blue number next to each edge in the following figure of the Weighted Graph. This number is utilized to signify the weight of the corresponding edge.

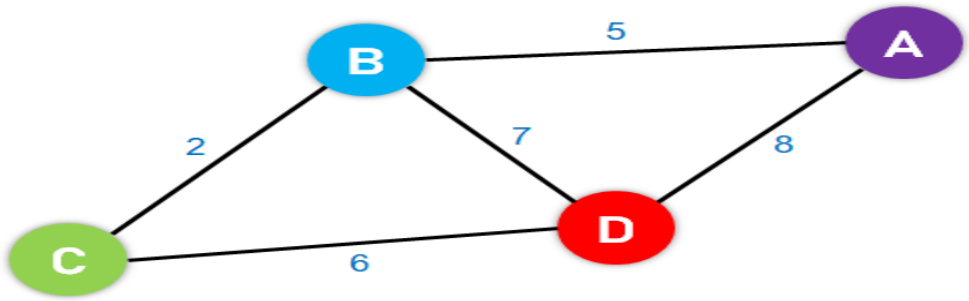


Figure 5: An Example of a Weighted Graph

An Introduction to Dijkstra's Algorithm

Now that we know some basic Graphs concepts let's dive into understanding the concept of Dijkstra's Algorithm.

Ever wondered how does Google Maps finds the shortest and fastest route between two places?

Well, the answer is **Dijkstra's Algorithm**. **Dijkstra's Algorithm** is a Graph algorithm **that finds the shortest path** from a source vertex to all other vertices in the Graph (single source shortest path). It is a type of Greedy Algorithm that only works on Weighted Graphs having positive weights. The time complexity of Dijkstra's Algorithm is $\mathbf{O(V^2)}$ with the help of the adjacency matrix representation of the graph. This time complexity can be reduced to **O((V + E) log V)** with the help of an adjacency list representation of the graph, where **V** is the number of vertices and **E** is the number of edges in the graph.

History of Dijkstra's Algorithm

Dijkstra's Algorithm was designed and published by **Dr. Edsger W. Dijkstra**, a Dutch Computer Scientist, Software Engineer, Programmer, Science Essayist, and Systems Scientist.

During an Interview with Philip L. Frana for the Communications of the ACM journal in the year 2001, Dr. Edsger W. Dijkstra revealed:

"What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city? It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty-minute invention. In fact, it was published in '59, three years later. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities. Eventually, that algorithm became to my great amazement, one of the cornerstones of my fame."

Dijkstra thought about the shortest path problem while working as a programmer at the Mathematical Centre in Amsterdam in 1956 to illustrate the capabilities of a new computer

known as ARMAC. His goal was to select both a problem and a solution (produced by the computer) that people with no computer background could comprehend. He developed the shortest path algorithm and later executed it for ARMAC for a vaguely shortened transportation map of 64 cities in the Netherlands (64 cities, so 6 bits would be sufficient to encode the city number). A year later, he came across another issue from hardware engineers operating the next computer of the institute: Minimize the amount of wire required to connect the pins on the machine's back panel. As a solution, he re-discovered the algorithm called Prim's minimal spanning tree algorithm and published it in the year 1959.

Fundamentals of Dijkstra's Algorithm

The following are the basic concepts of Dijkstra's Algorithm:

1. Dijkstra's Algorithm begins at the node we select (the source node), and it examines the graph to find the shortest path between that node and all the other nodes in the graph.
2. The Algorithm keeps records of the presently acknowledged shortest distance from each node to the source node, and it updates these values if it finds any shorter path.
3. Once the Algorithm has retrieved the shortest path between the source and another node, that node is marked as 'visited' and included in the path.
4. The procedure continues until all the nodes in the graph have been included in the path. In this manner, we have a path connecting the source node to all other nodes, following the shortest possible path to reach each node.

Understanding the Working of Dijkstra's Algorithm

A **graph** and **source vertex** are requirements for Dijkstra's Algorithm. This Algorithm is established on Greedy Approach and thus finds the locally optimal choice (local minima in this case) at each step of the Algorithm.

Each Vertex in this Algorithm will have two properties defined for it:

1. Visited Property
2. Path Property

The following is the step that we will follow to implement Dijkstra's Algorithm:

Step 1: First, we will mark the source node with a current distance of 0 and set the rest of the nodes to INFINITY.

Step 2: We will then set the unvisited node with the smallest current distance as the current node, suppose X.

Step 3: For each neighbor N of the current node X: We will then add the current distance of X with the weight of the edge joining X-N. If it is smaller than the current distance of N, set it as the new current distance of N.

Step 4: We will then mark the current node X as visited.

Step 5: We will repeat the process from '**Step 2**' if there is any node unvisited left in the graph.

Let us now understand the implementation of the algorithm with the help of an example:

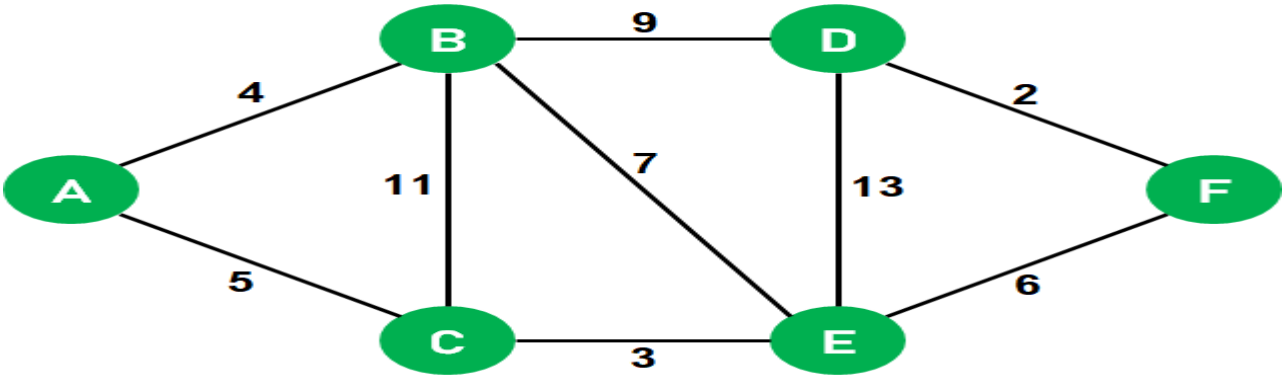


Figure 6: The Given Graph

1. We will use the above graph as the input, with node **A** as the source.
2. First, we will mark all the nodes as unvisited.
3. We will set the path to **0** at node **A** and **INFINITY** for all the other nodes.
4. We will now mark source node **A** as visited and access its neighboring nodes. **Note:** We have only accessed the neighboring nodes, not visited them.
5. We will now update the path to node **B** by **4** with the help of relaxation because the path to node **A** is **0** and the path from node **A** to **B** is **4**, and the **minimum((0 + 4), INFINITY)** is **4**.
6. We will also update the path to node **C** by **5** with the help of relaxation because the path to node **A** is **0** and the path from node **A** to **C** is **5**, and the **minimum((0 + 5), INFINITY)** is **5**. Both the neighbors of node **A** are now relaxed; therefore, we can move ahead.
7. We will now select the next unvisited node with the least path and visit it. Hence, we will visit node **B** and perform relaxation on its unvisited neighbors. After performing

relaxation, the path to node **C** will remain **5**, whereas the path to node **E** will become **11**, and the path to node **D** will become **13**.

8. We will now visit node **E** and perform relaxation on its neighboring nodes **B, D**, and **F**. Since only node **F** is unvisited, it will be relaxed. Thus, the path to node **B** will remain as it is, i.e., **4**, the path to node **D** will also remain **13**, and the path to node **F** will become **14 (8 + 6)**.
9. Now we will visit node **D**, and only node **F** will be relaxed. However, the path to node **F** will remain unchanged, i.e., **14**.
10. Since only node **F** is remaining, we will visit it but not perform any relaxation as all its neighboring nodes are already visited.
11. Once all the nodes of the graphs are visited, the program will end.

Hence, the final paths we concluded are:

1. A = 0
2. B = 4 (A -> B)
3. C = 5 (A -> C)
4. D = 4 + 9 = 13 (A -> B -> D)
5. E = 5 + 3 = 8 (A -> C -> E)
6. F = 5 + 3 + 6 = 14 (A -> C -> E -> F)

Pseudocode for Dijkstra's Algorithm

We will now understand a pseudocode for Dijkstra's Algorithm.

- We have to maintain a record of the path distance of every node. Therefore, we can store the path distance of each node in an array of size n, where n is the total number of nodes.
- Moreover, we want to retrieve the shortest path along with the length of that path. To overcome this problem, we will map each node to the node that last updated its path length.
- Once the algorithm is complete, we can backtrack the destination node to the source node to retrieve the path.

- We can use a minimum Priority Queue to retrieve the node with the least path distance in an efficient way.

Time and Space Complexity of Dijkstra's Algorithm

- The Time Complexity of Dijkstra's Algorithm is **O(E log V)**, where E is the number of edges and V is the number of vertices.
- The Space Complexity of Dijkstra's Algorithm is O(V), where V is the number of vertices.

Advantages and Disadvantages of Dijkstra's Algorithm

Let us discuss some advantages of Dijkstra's Algorithm:

1. One primary advantage of using Dijkstra's Algorithm is that it has an almost linear time and space complexity.
2. We can use this algorithm to calculate the shortest path from a single vertex to all other vertices and a single source vertex to a single destination vertex by stopping the algorithm once we get the shortest distance for the destination vertex.
3. This algorithm only works for directed weighted graphs, and all the edges of this graph should be non-negative.

Despite having multiple advantages, Dijkstra's algorithm has some disadvantages also, such as:

1. Dijkstra's Algorithm performs a concealed exploration that utilizes a lot of time during the process.
2. This algorithm is impotent to handle negative edges.
3. Since this algorithm heads to the acyclic graph, it cannot calculate the exact shortest path.
4. It also requires maintenance to keep a record of vertices that have been visited.

Some Applications of Dijkstra's Algorithm

Dijkstra's Algorithm has various real-world applications, some of which are stated below:

1. **Digital Mapping Services in Google Maps:** There are various times when we have tried to find the distance in Google Maps either from our location to the nearest preferred location or from one city to another, which comprises multiple routes/paths connecting them; however, the application must display the minimum distance. This is only possible because Dijkstra's algorithm helps the application find the shortest between two given locations along the path. Let us consider the USA as a graph wherein the cities/places are represented as vertices, and the routes between two cities/places are represented as edges. Then with the help of Dijkstra's Algorithm, we can calculate the shortest routes between any two cities/places.
2. **Social Networking Applications:** In many applications like Facebook, Twitter, Instagram, and more, many of us might have observed that these apps suggest the list of friends that a specific user may know. How do many social media companies implement this type of feature in an efficient and effective way, specifically when the system has over a billion users? The answer to this question is Dijkstra's Algorithm. The standard Dijkstra's Algorithm is generally used to estimate the shortest distance between the users measured through the connections or mutuality among them. When social networking is very small, it uses the standard Dijkstra's Algorithm in addition to some other features in order to determine the shortest paths. However, when the graph is much bigger, the standard algorithm takes several seconds to count, and thus, some advanced algorithms are used as the alternative.
3. **Telephone Network:** As some of us might know, in a telephone network, each transmission line has a bandwidth, 'b'. The bandwidth is the highest frequency that the transmission line can support. In general, if the frequency of the signal is higher in a specific line, the signal is reduced by that line. Bandwidth represents the amount of information that can be transmitted by the line. Let us consider a city a graph wherein the switching stations are represented using the vertices, the transmission lines are represented as the edges, and the bandwidth, 'b', is represented using the weight of the edges. Thus, as we can observe, the telephone network can also fall into the category of the shortest distance problem and can be solved using Dijkstra's Algorithm.
4. **Flight Program:** Suppose that a person requires software to prepare an agenda of flights for customers. The agent has access to a database with all flights and airports.

In addition to the flight number, origin airport, and destination, the flights also have departure and arrival times. So, in order to determine the earliest arrival time for the selected destination from the original airport and given start time, the agents make use of Dijkstra's Algorithm.

5. **IP routing to find Open Shortest Path First:** Open Shortest Path First (abbreviated as OSPF) is a link-state routing protocol used to find the best path between the source and destination router with the help of its own Shortest Path First. Dijkstra's Algorithm is extensively utilized in the routing protocols required by the routers in order to update their forwarding table. The algorithm gives the shortest cost path from the source router to the other routers present in the network.
6. **Robotic Path:** These days, drones and robots have come into existence, some operated manually and some automatically. The drones and robots which are operated automatically and used to deliver the packages to a given location or used for any certain task are configured with Dijkstra's Algorithm module so that whenever the source and destination are known, the drone and robot will move in the ordered direction by following the shortest path keeping the time taken to a minimum in order to deliver the packages.
7. **Designate the File Server:** Dijkstra's Algorithm is also used to designate a file server in a Local Area Network (LAN). Suppose that an infinite period of time is needed for the transmission of the files from one computer to another. So, to minimize the number of 'hops' from the file server to every other computer on the network, we will use Dijkstra's Algorithm. This algorithm will return the shortest path between the networks resulting in the minimum number of hops.

LECTURE 7

HUFFMAN CODES

- (i) Data can be encoded efficiently using Huffman Codes.
- (ii) It is a widely used and beneficial technique for compressing data.
- (iii) Huffman's greedy algorithm uses a table of the frequencies of occurrences of each character to build up an optimal way of representing each character as a binary string.

Suppose we have 10^5 characters in a data file. Normal Storage: 8 bits per character (ASCII) - 8 x 10^5 bits in a file. But we want to compress the file and save it compactly. Suppose only six characters appear in the file:

	a	b	c	d	e	f	Total
Frequency	45	13	12	16	9	5	100

How can we represent the data in a Compact way?

(i) Fixed length Code: Each letter represented by an equal number of bits. With a fixed length code, at least 3 bits per character:

For example:

a	000
b	001
c	010
d	011
e	100
f	101

For a file with 10^5 characters, we need 3 x 10^5 bits.

(ii) A variable-length code: It can do considerably better than a fixed-length code, by giving many characters short code words and infrequent character long codewords.

For example:

a	0

b 101

c 100

d 111

e 1101

f 1100

Number of bits = (45 x 1 + 13 x 3 + 12 x 3 + 16 x 3 + 9 x 4 + 5 x 4) x 1000

= 2.24 x 10^5bits

Thus, 224,000 bits to represent the file, a saving of approximately 25%.This is an optimal character code for this file.

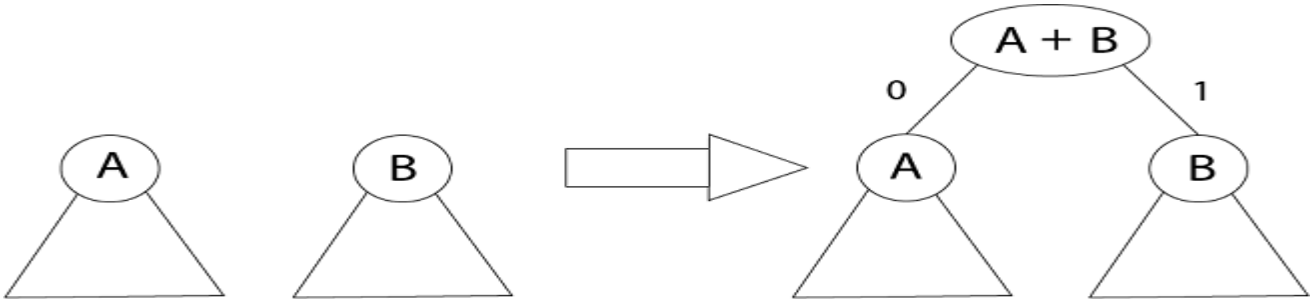
Prefix Codes:

The prefixes of an encoding of one character must not be equal to complete encoding of another character, e.g., 1100 and 11001 are not valid codes because 1100 is a prefix of some other code word is called prefix codes.

Prefix codes are desirable because they clarify encoding and decoding. Encoding is always simple for any binary character code; we concatenate the code words describing each character of the file. Decoding is also quite comfortable with a prefix code. Since no codeword is a prefix of any other, the codeword that starts with an encoded data is unambiguous.

Greedy Algorithm for constructing a Huffman Code:

Huffman invented a greedy algorithm that creates an optimal prefix code called a Huffman Code.



The algorithm builds the tree T analogous to the optimal code in a bottom-up manner. It starts with a set of |C| leaves (C is the number of characters) and performs |C| - 1 'merging' operations to create the final tree. In the Huffman algorithm 'n' denotes the quantity of a set of characters, z indicates the parent node, and x & y are the left & right child of z respectively.

LECTURE 8

AN ACTIVITY SELECTION PROBLEM

The activity selection problem is a mathematical optimization problem. Our first illustration is the problem of scheduling a resource among several challenge activities. We find a greedy algorithm provides a well designed and simple method for selecting a maximum- size set of manually compatible activities.

Suppose S = {1, 2....n} is the set of n proposed activities. The activities share resources which can be used by only one activity at a time, e.g., Tennis Court, Lecture Hall, etc. Each Activity "i" has **start time** s_i and a **finish time** f_i, where $s_i \leq f_i$. If selected activity "i" take place meanwhile the half-open time interval $[s_i, f_i)$. Activities i and j are **compatible** if the intervals (s_i, f_i) and $[s_i, f_i)$ do not overlap (i.e. i and j are compatible if $s_i \geq f_i$ or $s_i \geq f_i$). The activity-selection problem chosen the maximum- size set of mutually consistent activities.

Algorithm Of Greedy- Activity Selector:

GREEDY- ACTIVITY SELECTOR (s, f)

1. n ← length [s]
2. A ← {1}
3. j ← 1.
4. for i ← 2 to n
5. do if $s_i \geq f_i$
6. then A ← A ∪ {i}
7. j ← i
8. return A

Example: Given 10 activities along with their start and end time as

S = (A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A9 A10)

Si = (1,2,3,4,7,8,9,9,11,12)

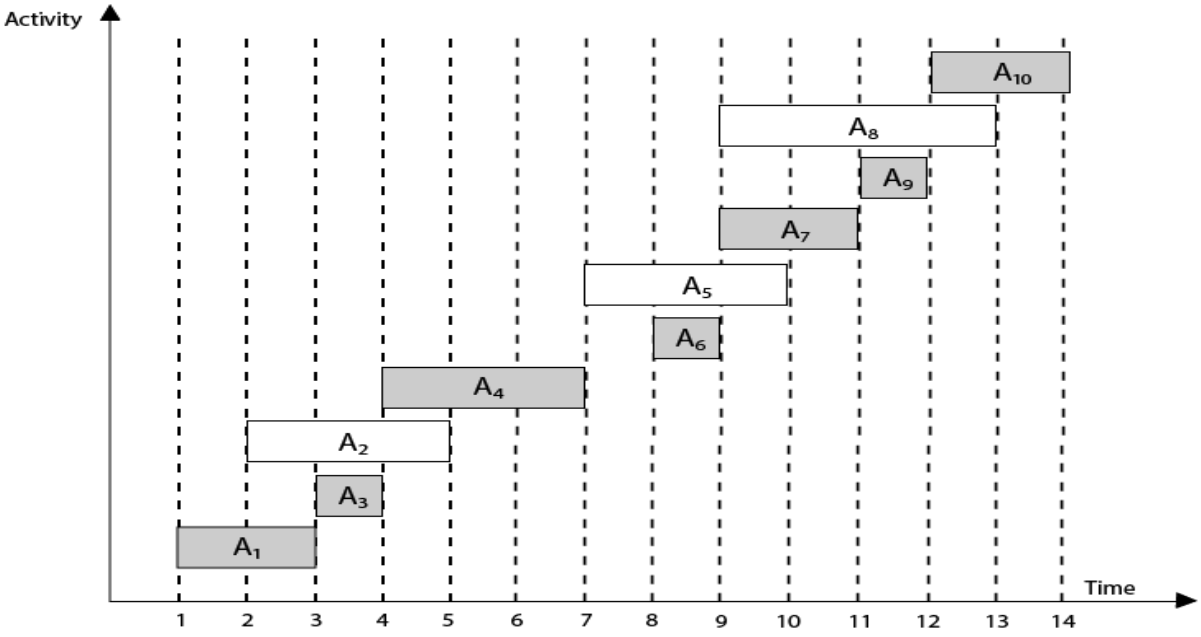
fi = (3,5,4,7,10,9,11,13,12,14)

Compute a schedule where the greatest number of activities takes place.

Solution: The solution to the above Activity scheduling problem using a greedy strategy is illustrated below:

Arranging the activities in increasing order of end time

Activity	A_1	A_3	A_2	A_4	A_6	A_5	A_7	A_9	A_8	A_{10}
Start	1	3	2	4	8	7	9	11	9	12
Finish	3	4	5	7	9	10	11	12	13	14



Now, schedule A_1

Next schedule A_3 as A_1 and A_3 are non-interfering.

Next **skip** A_2 as it is interfering.

Next, schedule A_4 as A_1 A_3 and A_4 are non-interfering, then next, schedule A_6 as A_1 A_3 A_4 and A_6 are non-interfering.

Skip A_5 as it is interfering.

Next, schedule A_7 as A_1 A_3 A_4 A_6 and A_7 are non-interfering.

Next, schedule A_9 as A_1 A_3 A_4 A_6 A_7 and A_9 are non-interfering.

Skip A_8 as it is interfering.

Next, schedule A_{10} as A_1 A_3 A_4 A_6 A_7 A_9 and A_{10} are non-interfering.

Thus the final Activity schedule is:

$(A_1\ A_3\ A_4\ A_6\ A_7\ A_9\ A_{10})$